



CAHIER DES CHARGES

VIRAL STICK

Générateur de Mèmes & Stickers IA Multimodal

De l'exercice académique ICT202 (Générateur de Mèmes Multimodal)
à la plateforme sociale complète de stickers IA — vision KERNEL FORGE

Porteur de projet : NGHOMSI FEUKOUO Ravel (« Archlord »)

Collectif : KERNEL FORGE

Établissement : Université de Yaoundé I — Licence ICT4D, parcours Génie Logiciel — ICT202
(G1 / G2)

Date : 20 juin 2026

Version 2.0 — Document de travail soumis à validation avant développement

Table des matières

1. Objet du document

Ce cahier des charges consolide en un seul document de référence les trois sources transmises pour le projet Viral Stick, conformément à la consigne reçue : produire d'abord un cahier des charges complet, à valider, avant tout développement.

- Le sujet officiel du TP « Générateur de Mêmes Multimodal » (ICT202, groupes G1 et G2) — le socle académique évalué et son délai.
- Les précisions complémentaires transmises par message — la vision produit étendue (filtres/effets façon CapCut, génération de stickers par IA, deepfake visage/voix, illustration personnalisée à la demande).
- La liste détaillée de tâches à accomplir — l'ensemble des exigences fonctionnelles, techniques, UX, de déploiement et de structuration du projet.

Le document distingue systématiquement ce qui relève du périmètre académique strict (Core/Bonus du TP, délai d'une semaine) de ce qui relève de la vision produit complète portée par KERNEL FORGE (Viral Stick en tant que plateforme). Une section dédiée (§16) signale les points qui nécessitent une clarification ou une décision avant le lancement du développement.

Version	Date	Changements
1.0	19/06/2026	Document initial, consolidation des trois sources de départ (sujet du TP, contexte étendu, liste de tâches).
2.0	20/06/2026	Intégration du roster définitif des 7 compagnons (avatars + rôles assignés par page/service, cf. §7.4) ; prise en compte des dossiers mobile/ et web/ déjà initialisés par le porteur de projet dans le dépôt (cf. §11, règle de non-suppression).

2. Contexte et enjeux

2.1 Contexte académique (TP ICT202)

Avec l'essor des réseaux sociaux et des messageries instantanées (WhatsApp, TikTok, Telegram), la communication numérique s'appuie de plus en plus sur l'image, l'audio et le contenu humoristique (« mêmes ») plutôt que sur le seul texte. En parallèle, les API d'intelligence artificielle (génération de texte, reconnaissance vocale, analyse d'image) rendent possible la création de contenu dynamique et contextualisé en temps réel.

L'exercice demande de concevoir une application mobile agissant comme un Générateur de Mêmes Multimodal : elle doit capter le contexte d'une discussion ou d'un média (texte, audio, image) et utiliser des API d'IA pour générer automatiquement un contenu humoristique prêt à être partagé.

Le sujet concerne les deux groupes ICT202 G1 et G2, avec un délai annoncé d'une semaine.

2.2 Vision produit étendue : Viral Stick

Au-delà du strict exercice académique, le porteur de projet souhaite faire de ce TP la base d'une véritable application — Viral Stick — un studio de création de mêmes et de stickers

personnalisés par IA, dans l'esprit de CapCut pour l'édition (filtres, effets, calques de texte) et des packs de stickers WhatsApp pour la diffusion.

La fonctionnalité différenciante évoquée dans les échanges (« la patate de l'application ») est la génération d'illustrations personnalisées à la demande : l'utilisateur fournit un texte ou une situation, et l'application génère une illustration mettant en scène son propre avatar/visage dans cette situation — sur le même principe que les exemples de mèmes fournis en référence (cf. §17.1), où un visuel type est réutilisé avec un texte différent à chaque fois.

Point de vigilance — personnalisation par image/voix

Les documents transmis évoquent la génération de stickers « en deepfakant les visages, les voix etc. ». Ce cahier des charges encadre cette fonctionnalité comme une création d'avatar personnel : elle doit s'appuyer sur la photo/voix propre de l'utilisateur, avec son consentement explicite, et ne doit pas permettre de générer un contenu représentant un tiers sans l'accord de celui-ci. Ce point est repris en détail au §16.

3. Objectifs

3.1 Objectifs pédagogiques (cadre du TP)

- Développer une application mobile cross-platform fluide et réactive avec React Native.
- Concevoir et déployer une API backend légère avec Express.js (Node.js) pour orchestrer les requêtes.
- Manipuler des flux multimédias : capture photo, enregistrement audio, upload et manipulation de fichiers.
- Intégrer des services d'intelligence artificielle (NLP, Speech-to-Text, Computer Vision).
- Gérer les interactions avec le système d'exploitation mobile (partage de fichiers, permissions caméra/micro).

3.2 Objectifs produit (vision KERNEL FORGE)

- Livrer une application mobile ET une version web, chacune avec son panel d'administration intégré.
- Offrir une expérience ludique, fun et conviviale, animée, pensée pour un public jeune (« meme »).
- Construire une brique de génération IA multimodale réutilisable (texte, audio, image) reposant uniquement sur des services à offre gratuite (Gemini, Mistral, Ollama, OpenRouter).
- Créer une dynamique communautaire autour des créations (forum public, classement des créateurs).
- Faciliter la diffusion virale des stickers, en ligne et hors-ligne, y compris vers WhatsApp.

4. Glossaire

Terme	Définition dans le cadre du projet
Context Reader	Module Core (texte) : l'utilisateur colle un extrait de discussion ; l'IA en analyse le ton et génère un mème adapté (image et/ou texte).
Voice-to-Meme	Module Core (audio) : note vocale transcrite (Speech-to-Text), analysée, puis transformée en mème avec sous-titre.
Status Remixer	Module Core (image) : image importée par l'utilisateur, enrichie de texte généré par IA et/ou de retouches visuelles basiques.
Sticker personnalisé	Illustration générée à la demande mettant en scène l'avatar/visage propre de l'utilisateur dans une situation décrite par un texte.
Pack de stickers	Ensemble de stickers regroupés, partageable en un seul bloc (y compris vers WhatsApp).
Forum	Espace public où les stickers/mèmes créés sont visibles par tous, et peuvent être utilisés ou « remixés » (copiés puis modifiés) par d'autres utilisateurs.
Compagnon	Mascotte animée affichée en coin d'écran, qui dialogue par bulles de texte (humour, état d'avancement, encouragements).
Drawer Navigation Animation	Transition animée entre l'écran principal et le menu latéral, combinant translation, mise à l'échelle, rotation/inclinaison et masquage à bords arrondis.
Deep link	Lien (https) permettant d'ouvrir directement un contenu précis (sticker, pack) dans l'application depuis l'extérieur.
MVC	Architecture Modèle-Vue-Contrôleur imposée pour structurer le code backend, mobile et web.

5. Périmètre fonctionnel

5.1 Fonctionnalités Cœur — exigées par le TP (Core)

- Context Reader (texte) : saisie/collage d'un extrait de discussion → analyse du ton et de la situation côté backend → génération d'une image ou d'un texte de mème adapté.
- Voice-to-Meme (audio) : enregistrement d'une note vocale → transcription Speech-to-Text → analyse de l'émotion/contenu → proposition d'image humoristique avec la transcription en sous-titre.
- Status Remixer (image) : import d'une image du téléphone → ajout de texte généré par IA et/ou retouches visuelles basiques pour produire un mème.

5.2 Fonctionnalités Bonus du TP

- Réception directe d'image ou de texte depuis une autre application via le Share Intent du système (ex. depuis WhatsApp).

- Génération d'image par IA (modèle de diffusion) pour créer le visuel du même à partir de zéro selon le contexte de la discussion.
- Localisation culturelle : adaptation de l'humour avec des expressions ou références locales.

5.3 Fonctionnalités étendues — vision produit complète (au-delà du TP)

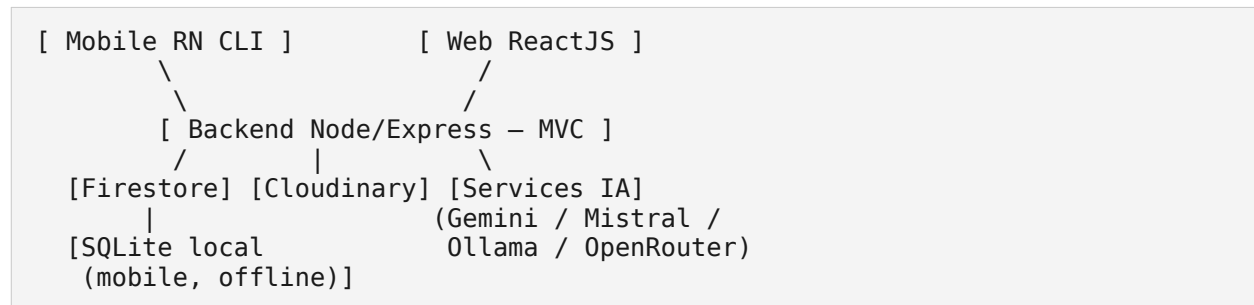
- Génération de sticker personnalisé : à partir d'un texte/situation, génération d'une illustration mettant en scène l'avatar de l'utilisateur (sous réserve du cadrage défini au §16).
- Édition avancée façon CapCut : filtres, effets visuels, calques de texte animés sur les créations.
- Forum public : publication, like, commentaire, et « dupliquer pour remixer » les créations des autres utilisateurs.
- Classement (leaderboard) des meilleurs créateurs, basé sur les likes reçus en forum, les commentaires reçus et le nombre de partages réussis.
- Partage de stickers et de packs : en ligne via deep link https, et hors-ligne via Bluetooth.
- Intégration WhatsApp pour ajouter directement les stickers créés au clavier de stickers WhatsApp.
- Compagnon IA interactif en coin d'écran, avec plusieurs personnages disponibles (dossier /asset/compagnons).
- Panel d'administration intégré à chaque version (mobile et web).
- Page « À propos » présentant le projet, son lien GitHub et la fiche de l'équipe.
- Deux thèmes graphiques complets, chacun avec sa propre palette de couleurs cohérente (y compris les espaces vides).
- Écran de démarrage animé, dynamique et unique, d'une durée fixe de 6 secondes, avec le logo.

6. Architecture technique

6.1 Vue d'ensemble

La stack JavaScript/TypeScript est imposée par le sujet académique et reprise pour la version produit complète :

- Mobile : React Native CLI (Community), avec gestion correcte des zones sécurisées (SafeAreaView).
- Web : ReactJS avec frameworks associés, conçu pour respecter les limites du plan Vercel gratuit.
- Backend (API Gateway) : Node.js + Express.js, architecture MVC, gestion des uploads (Multer), centralisation et sécurisation des appels vers les API d'IA externes.
- Chaque version (mobile, web) embarque son propre panel d'administration.
- Stockage des données : Firestore (NoSQL, distant) + SQLite (local, persistance hors-ligne côté mobile).
- Stockage des médias : Cloudinary.



6.2 Modèles et services IA proposés

Contrainte exprimée par le porteur de projet : n'utiliser que des comptes gratuits. Les quotas ci-dessous évoluent fréquemment selon les fournisseurs ; ils sont indicatifs et doivent être revérifiés dans les consoles officielles avant chaque jalon de développement (voir aussi §16-c).

Besoin	Service proposé	Usage	Remarque plan gratuit
Compréhension de texte / génération de même (Context Reader)	Google Gemini (modèles Flash / Flash-Lite, via Google AI Studio)	NLP texte	Sans carte bancaire ; quotas par minute/jour modestes et variables, à surveiller.
Repli / texte volumineux	Mistral AI — offre « Experiment »	NLP texte	Sans carte bancaire (vérification téléphone) ; volume mensuel de tokens généreux pour le prototypage.
Repli multi-fournisseurs	OpenRouter (modèles suffixés « :free »)	NLP texte	Gratuit mais partagé et limité en débit ; prévoir une bascule automatique vers un autre modèle « :free » en cas d'erreur 429.
Tests locaux / vie privée / sans quota	Ollama (modèles open-weights hébergés en local)	NLP texte	Aucune limite d'appel, mais dépend des ressources matérielles disponibles ; adapté au développement, pas forcément à la production.
Transcription audio (Voice-to-Meme)	Whisper (open-weights), via Hugging Face Inference ou exécution locale	Speech-to-Text	Tier gratuit limité en débit ; auto-hébergement possible si besoin de volume.
Voix du Compagnon (texte → parole)	Piper TTS (open-source, déjà utilisé sur le projet J.A.R.V.I.S.) ou modèles TTS Hugging Face	Text-to-Speech	Auto-hébergement gratuit et illimité (Piper) ou tier gratuit limité (HF).
Génération d'image (Status Remixer / sticker personnalisé)	Modèles de diffusion via Hugging Face Inference Providers	Texte → Image	Tier gratuit avec quotas et possible temps de démarrage à froid ; à tester avant de s'y appuyer en production.

6.3 Bases de données

6.3.1 Firestore (distant, NoSQL)

Schéma de collections proposé pour le transit des informations et le stockage des messages/contenus :

Collection	Champs clés (proposition)
users	id, pseudo, avatarUrl, theme, score, dateCreation
stickers	id, ownerId, imageUrl, texte, themeStyle, dateCreation, visibilite (privé/forum)
packs	id, ownerId, listeStickerIds, dateCreation
forum_posts	id, stickerId, ownerId, nbLikes, nbCommentaires, nbPartages, dateCreation
comments	id, postId, authorId, texte, dateCreation
likes	id, postId, userId, dateCreation
shares	id, stickerId, canal (lien/bluetooth/whatsapp), dateCreation
rankings	userId, scoreCalcule, dateMiseAJour (vue dérivée, recalculée périodiquement)

Ce schéma est une proposition de structuration initiale ; il sera affiné lors de la conception technique détaillée (hors périmètre du présent cahier des charges).

6.3.2 SQLite (local, mobile)

Base locale pour la persistance hors-ligne, afin de ne pas dépendre uniquement de Firestore :

- stickers_cache : copie locale des stickers de l'utilisateur et des derniers contenus consultés.
- drafts : créations en cours, non encore synchronisées.
- settings : préférences (thème actif, compagnon choisi).

À spécifier en conception technique

La stratégie de synchronisation entre Firestore et SQLite (différée, résolution de conflits, mode hors-ligne complet) reste à détailler lors de la phase de conception technique ; ce cahier des charges en fixe l'exigence sans en imposer l'implémentation exacte.

6.4 Stockage des médias — Cloudinary

Toutes les images et fichiers qui transitent par le projet (photos importées, stickers générés, avatars, packs) sont stockés sur Cloudinary, qui assure également les transformations basiques (redimensionnement, recadrage, filtres serveur).

6.5 Système de prompt IA

Le system prompt est un fichier JSON complet, embarqué directement dans le code des versions web et mobile, adapté au contexte et à la nature du projet. Il doit être conçu pour être le plus cohérent et performant possible, et capable de comprendre les blagues, le second degré et les mêmes.

- Structure proposée : persona (ton du générateur), règles de ton, contexte culturel/local, exemples few-shot, garde-fous de modération.
- Le contenu précis de ce fichier sera rédigé lors du développement, en cohérence avec les exemples de mêmes fournis en référence (§17.1).

7. UX / UI & Design System

7.1 Système à deux thèmes

L'application doit proposer deux thèmes graphiques complets (interfaces « closes », chacune cohérente de bout en bout), chacun avec sa propre palette de couleurs — y compris pour les espaces vides, afin d'éviter toute incohérence visuelle entre les écrans d'un même thème.

7.2 Écran de démarrage

Écran de chargement dynamique et unique, avec animation et logo, d'une durée fixe de 6 secondes. Les appels API doivent être organisés pour tirer parti de cette fenêtre de temps (préchargement, initialisation) plutôt que d'ajouter une attente supplémentaire après l'écran de chargement.

7.3 Animation de navigation (menu ⇌ accueil)

Le document transmis décrit précisément l'effet attendu, qu'il nomme « Advanced Drawer Navigation Animation », combinant quatre composantes synchronisées :

1. Translation : l'écran principal glisse horizontalement pour révéler le menu en arrière-plan.
2. Mise à l'échelle : l'écran principal rétrécit légèrement pour donner une impression de profondeur.
3. Rotation / inclinaison : un léger skew crée un effet de perspective 3D.
4. Masquage à bords arrondis : des coins arrondis apparaissent sur l'écran principal pendant le mouvement, renforçant l'effet de « feuille » qui se soulève.

Durée de transition cible : 300 ms, avec une courbe d'animation de type ease-in-out.

Incohérence détectée — à confirmer

La documentation technique fournie pour cette animation référence le package Flutter « flutter_advanced_drawer » et un fichier lib/main.dart, ainsi qu'une « application Papo ». Ces éléments correspondent au projet PayPoint (PAPO), qui est en Flutter, et non à Viral Stick, dont la stack imposée est React Native CLI. Ce fragment semble provenir d'un autre document par erreur.

Pour Viral Stick, l'effet visuel décrit (translation + scale + rotation + clipping arrondi, 300 ms, ease-in-out) sera repris comme exigence fonctionnelle, mais implémenté avec les outils React Native équivalents (par ex. react-native-reanimated + react-native-gesture-handler, ou une librairie de drawer animé compatible RN CLI). À confirmer avec le porteur de projet avant développement.

La couleur affichée dans les zones vides révélées pendant la transition doit correspondre au thème actif, pour éviter tout flash de couleur incohérente.

7.4 Compagnon interactif

Le système de compagnons a été précisé par le porteur de projet : ce n'est plus une mascotte unique, mais un roster de 7 avatars, chacun assigné à une page ou un service précis de l'application, dans une logique d'organisation interne (« on peut considérer toute cette application comme une entreprise »). Chaque compagnon accueille et dialogue avec l'utilisateur sur son périmètre dédié.

Avatar	Nom (fichier) — Rôle assigné
	Archlord (arch.png) — administrateur système et responsable global de l'application (rôle de « PDG »). Vocation à intervenir de façon transverse, en supervision, potentiellement sur l'ensemble des pages.
	Para (para.png) — gère la page Paramètres : c'est lui qui accueille l'utilisateur dès l'ouverture de cette page et qui échange avec lui sur les réglages de l'application.
	Secu (secu.png) — avatar dédié à la sécurité. Il apparaît lors d'erreurs de sécurité ou de toute action liée à la sécurité, et est présent en permanence sur la page Sécurité.
	Data (data.png) — en charge de la page Données utilisateur et de l'assistance/support ; intervient aussi sur la page de chat système pour guider les utilisateurs perdus ou qui cherchent à contacter le support.
	Bio (bio.png) — l'un des 3 compagnons « artistes » qui couvrent ensemble toutes les autres pages de l'application (non assignées à un compagnon dédié).
	Ubu (ubu.png) — second compagnon « artiste » du trio ; intervient ponctuellement sur les pages restantes.
	Art (art.png) — troisième compagnon « artiste » du trio. Les 3 artistes (Bio, Ubu, Art) viennent de temps en temps sur leurs pages communiquer avec l'utilisateur, raconter des blagues, et peuvent aussi dialoguer entre eux.

Exigences complémentaires exprimées sur ce module :

- Le système d'apparition et de dialogue des compagnons doit être conçu pour rendre l'application la plus interactive possible (logique de déclenchement contextuel à approfondir en conception détaillée : ouverture de page, erreur, inactivité, étape clé d'un traitement IA, etc.).

- Une version animée (GIF) de chaque compagnon est prévue à terme, en complément de l'image statique fournie ici — à produire/intégrer lors d'un lot ultérieur.
- Les 3 compagnons « artistes » (Bio, Ubu, Art) doivent pouvoir être mis en scène en train de converser entre eux, et pas seulement avec l'utilisateur.

8. Fonctionnalités sociales détaillées

8.1 Forum public

Flux public où l'ensemble des stickers et mèmes créés sont visibles par tous les utilisateurs. Chacun peut « utiliser » une création existante ou la « remixer » (copie suivie d'une modification personnelle), avec like et commentaires sur chaque publication.

8.2 Partage en ligne et hors-ligne

- En ligne : partage via lien profond (deep link) en https, ouvrant directement le sticker ou le pack concerné dans l'application.
- Hors-ligne : partage via Bluetooth entre appareils proches, sans connexion internet.

8.3 Classement (leaderboard)

Un classement des meilleurs créateurs de stickers est calculé à partir de trois signaux : les likes reçus sur les créations publiées en forum, les commentaires reçus, et le nombre de partages réussis. La formule exacte de pondération entre ces trois signaux reste à valider (cf. §16-f).

8.4 Intégration WhatsApp

Les stickers créés doivent pouvoir être ajoutés directement à WhatsApp, sans friction excessive pour l'utilisateur (mécanisme standard d'ajout de pack de stickers WhatsApp sur Android).

9. Panel d'administration

Un panel d'administration est inclus dans chaque version (mobile et web). Fonctions attendues à ce stade :

- Modération du forum (contenus signalés, suppression).
- Gestion des comptes utilisateurs.
- Supervision des quotas/usage des différents services IA (Gemini, Mistral, Ollama, OpenRouter).
- Statistiques d'usage global (créations, partages, classement).
- Gestion des compagnons et des thèmes disponibles.

10. Page « À propos »

Présente à la fois sur la version mobile et sur la version web, cette page doit afficher :

- Le lien du dépôt GitHub du projet.

- Le thème du projet, son rôle et son utilité.
- La liste des membres de l'équipe ayant travaillé sur le projet.

Pour chaque membre, les éléments suivants sont requis :

5. Nom du membre.
6. Rôle dans le projet.
7. Lien vers sa photo / photo de profil.
8. Pseudo GitHub, sous forme de lien vers sa page GitHub.

11. Structure du projet (arborescence proposée)

Règle impérative — dossiers déjà initialisés par le porteur de projet

Les dossiers mobile/ et web/ existent déjà à la racine du dépôt : ils ont été créés manuellement par le porteur de projet lui-même comme point de départ. Ces dossiers, ainsi que tout document qu'il y crée au fil du projet, ne doivent jamais être supprimés ni recréés depuis zéro lors du développement ou d'un scaffolding automatisé.

Toute génération de code ou de structure doit s'insérer dans l'arborescence existante (ajout/complément), jamais par suppression-puis-recréation des dossiers de premier niveau.

```
viral-stick/
├── mobile/           React Native CLI Community (SafeAreaView)
│   └── src/          (MVC, 1 fichier/écran) [existant – ne pas supprimer]
├── web/             ReactJS + framework(s) – limites Vercel free
│   └── src/          (MVC, 1 fichier/écran) [existant – ne pas supprimer]
├── backend/         Node.js + Express – architecture MVC
│   ├── models/
│   ├── controllers/
│   ├── routes/
│   └── services-ia/
├── asset/
│   ├── logo/
│   └── compagnons/   (arch, para, secu, data, bio, ubu, art,
│                       compagnon.txt)
├── docs/
├── .github/workflows/ (CI/CD : build APK / AAB)
└── vercel.json
```

Méthodologie de présentation du code : architecture MVC pour les trois modules (mobile, web, backend), afin de faciliter la compréhension et la maintenance. Chaque écran ou page doit disposer de son propre fichier, pour éviter les fichiers de code trop longs et difficiles à maintenir.

12. Déploiement et intégration continue

12.1 Version web → Vercel

La version web (avec son panel admin) est déployée sur Vercel depuis le dépôt du projet, via un fichier vercel.json à configurer.

Point de vigilance — limites du plan Vercel gratuit

Les fonctions serverless du plan gratuit ont un temps d'exécution limité (de l'ordre de quelques secondes), ce qui peut être insuffisant pour certains appels IA plus longs (génération d'image ou de voix). Il est recommandé de prévoir un traitement asynchrone (file d'attente + récupération du résultat par sondage ou webhook) plutôt qu'un appel bloquant pour ces opérations.

12.2 Version mobile → GitHub Actions

La version mobile (avec son panel admin) est également hébergée dans le dépôt, avec des GitHub Actions dédiées à la construction de l'APK de release et de l'AAB (pour publication sur store).

12.3 Variables d'environnement — environnement GitHub « STICK »

Un environnement GitHub nommé STICK regroupe les variables nécessaires aux builds :

Variable	Contenu
GOOGLE_SERVICE_ACCOUNT	Contenu complet du fichier google.json (accès à la base NoSQL Firestore).
CLOUDINARY_CLOUD_NAME / API_KEY / API_SECRET	Accès au stockage des fichiers/images transitant par le projet.
GEMINI_API_KEY	Clé d'accès Google AI Studio (Gemini).
MISTRAL_API_KEY	Clé d'accès à l'offre gratuite Mistral.
OPENROUTER_API_KEY	Clé d'accès aux modèles gratuits OpenRouter.
OLLAMA_BASE_URL	URL de l'instance Ollama si un hébergement local/dédié est utilisé en environnement de build/test.

13. Livrables attendus

13.1 Livrables académiques (TP)

1. Code source : dépôts Git (frontend et backend) avec code commenté et historique de commits démontrant le travail d'équipe.
2. Documentation (README) : instructions claires pour installer et lancer le projet en local (frontend et backend), ainsi que la configuration des clés d'API nécessaires.
3. Démonstration vidéo : 3 à 5 minutes, montrant les parcours utilisateurs (Voice-to-Meme, Context Reader) sur téléphone physique ou émulateur.

13.2 Livrables additionnels — vision produit

- Documentation de l'animation de navigation (Drawer) et de son adaptation React Native.
- Schéma de base de données (Firestore + SQLite).
- Fichier JSON du system prompt, documenté.
- Fiche équipe pour la page « À propos ».

14. Méthodologie de développement

- Architecture MVC appliquée de façon homogène sur le backend, le mobile et le web.
- Un fichier dédié par écran/page, sur les trois modules, pour limiter la taille des fichiers et faciliter la maintenance.
- Stack JavaScript/TypeScript imposée, cohérente entre le mobile (React Native CLI) et le web (ReactJS).
- Sécurisation des appels IA externes centralisée côté backend (clés jamais exposées côté client).

15. Phasage proposé

L'échéance académique communiquée (« jusqu'à 1 semaine ») couvre par construction un périmètre beaucoup plus restreint que l'ensemble des exigences listées dans le document « travail à faire » (forum, classement, partage Bluetooth, intégration WhatsApp, deux thèmes, panels admin, CI/CD, double base de données, etc.). Le phasage suivant est proposé pour rendre le projet réalisable par étapes :

Lot 1 — MVP académique (délai TP, ≈ 1 semaine)

- Context Reader + Voice-to-Meme + Status Remixer (Core).
- Backend Express minimal, un seul fournisseur IA (ex. Gemini Flash) pour limiter les risques de quota.
- README + vidéo de démonstration (3-5 min).

Lot 2 — V1 produit KERNEL FORGE

- Deux thèmes, écran de démarrage 6 s, compagnon interactif, panel admin.
- Mise en place Firestore + SQLite, déploiement Vercel (web) et premières GitHub Actions (mobile).

Lot 3 — V2, vision complète

- Forum public, classement, partage deep link + Bluetooth, intégration WhatsApp.
- Génération de sticker personnalisé (avatar), multi-fournisseurs IA avec bascule automatique, animation Drawer avancée.

Recommandation

Valider ce phasage (ou un découpage alternatif) avec le porteur de projet avant de démarrer le développement, afin d'aligner le livrable du Lot 1 avec le délai académique réel, tout en gardant le cap sur la vision complète portée par KERNEL FORGE pour les lots suivants.

16. Points de vigilance à valider

- a) Incohérence de stack sur l'animation Drawer : la documentation fournie référence Flutter (`flutter_advanced_drawer`, `lib/main.dart`, « application Papo »), alors que Viral Stick est en React Native CLI. Voir détail et proposition au §7.3.

- **b) Personnalisation par visage/voix** : à cadrer strictement à l'usage du visage/de la voix propre de l'utilisateur, avec consentement explicite, sans génération de contenu représentant un tiers sans son accord. Voir §2.2.
- **c) Quotas des IA gratuites** : les chiffres communiqués par Gemini, Mistral, OpenRouter et Hugging Face évoluent fréquemment et varient selon les sources ; à révérifier en direct dans les consoles officielles avant chaque jalon, avec une logique de bascule entre fournisseurs en cas d'erreur de quota (429).
- **d) Limites du plan Vercel gratuit** : timeout des fonctions serverless potentiellement trop court pour certains appels IA (image/voix) — prévoir un traitement asynchrone. Voir §12.1.
- **e) Stratégie de synchronisation Firestore $\nrightarrow$ SQLite local** : non détaillée dans ce cahier des charges, à spécifier lors de la conception technique. Voir §6.3.2.
- **f) Formule exacte du classement** (pondération likes / commentaires / partages) à valider avec le porteur de projet. Voir §8.3.
- **g) Préservation impérative des dossiers mobile/ et web/ déjà initialisés** par le porteur de projet : aucune suppression ni recréation depuis zéro lors du développement. Voir §11.

17. Annexes

17.1 Exemples de mèmes fournis (référence de ton et de format)

Trois images ont été fournies comme référence du ton et du format attendus. Elles partagent une même structure : un visuel figé (deux personnages transportant une civière), réutilisé à chaque fois avec des bulles de texte différentes, sur le registre de l'humour étudiant local (notes, niveaux d'étude, situations vécues en faculté). Ce type de structure — un même visuel, un texte qui change — correspond exactement à la logique attendue du Context Reader et du Status Remixer : générer rapidement une légende adaptée à un contexte donné, plutôt que de redessiner systématiquement un visuel inédit.

17.2 Fiche équipe (modèle à compléter)

Nom	Rôle	Photo / Avatar	Pseudo GitHub (lien)

17.3 Sources consolidées dans ce document

- theme_du_tp.pdf — sujet officiel du TP ICT202 (G1/G2).
- contex_en_plus_du_theme — précisions complémentaires sur la vision produit.
- travail_à_faire.txt — liste détaillée des tâches et exigences.
- images_exemple_test/ — trois exemples de mèmes de référence.
- asset/logo/logo.jpg — logo du projet.

- `asset/compagnons/` (v2) — `arch.png`, `para.png`, `secu.png`, `data.png`, `bio.png`, `ubu.png`, `art.png` et `compagnon.txt` (rôles détaillés, cf. §7.4).
- `mobile/` et `web/` — dossiers déjà initialisés par le porteur de projet dans le dépôt (v2), à préserver (cf. §11).